

RAII — Խորը ուղեցույց

Resource Acquisition Is Initialization

Այս ուղեցույցը մանրամասն բացատրում է RAII-ը՝ ինչ է, ինչպես է աշխատում, ինչու է C++-ի ամենակարևոր idiom-ներից, իրական օրինակներով (memory, file, lock, mutex), և հարցազրույցի հարցերով:

1. Ի՞նչ է RAII-ը

RAII = **Resource Acquisition Is Initialization**. C++-ի հիմնական idiom, որտեղ resource-ը (memory, file, lock, socket) **կապվում է object-ի կյանքի հետ**:

Resource-ը վերցվում է -> constructor-ում (object ստեղծվելիս)

Resource-ը ազատվում է -> destructor-ում (object ոչնչանալիս)

Object-ը scope-ից դուրս գալիս -> destructor ավտոմատ կանչվում -> resource ավտոմատ ազատվում: **Չկա մոռացում, չկա leak**:

2. Խնդիրը առանց RAII

```
void f() {
    int* p = new int[100];
    FILE* file = fopen("data.txt", "r");

    if (error) return;    // LEAK -- delete[] ու fclose չեն կանչվում
    mayThrow();           // exception -> նույն խնդիրը

    delete[] p;
    fclose(file);
}
```

Ամեն exit path-ում (return, exception, break) պետք է ձեռքով ազատել -> հեշտ է մոռանալ -> leak / resource leak:

3. RAII-ով լուծումը

```
class IntArray {
    int* data;
public:
    IntArray(int size) { data = new int[size]; }    // վերցնում ctor-ում
    ~IntArray() { delete[] data; }                 // ազատում dtor-ում
};

void f() {
    IntArray arr(100);
    if (error) return;    // ոչ մի leak -- dtor ավտոմատ կանչվում
    mayThrow();           // exception -> stack unwinding -> dtor կանչվում
}                          // scope-ի վերջ -> dtor -> delete[]
```

Կարևորը՝ որ աշխատում է նույնիսկ exception-ի դեպքում: **stack unwinding** — exception-ի ժամանակ C++-ն ավտոմատ կանչում է բոլոր local object-ների destructor-ները:

4. RAII իրական օրինակներ

4.1 Memory (smart pointer)

```
{
    auto p = std::make_unique<int>(5);
}    // ավտոմատ delete
```

4.2 File

```
class File {
    FILE* f;
public:
    File(const char* name) { f = fopen(name, "r"); }
    ~File() { if (f) fclose(f); }    // ավտոմատ close
};

void read() {
    File file("data.txt");
    // ... օգտագործում
}    // ավտոմատ fclose, նույնիսկ exception-ի դեպքում
```

4.3 Mutex / Lock (ամենակարևոր)

```
std::mutex mtx;

void safe() {
    std::lock_guard<std::mutex> lock(mtx);    // ctor -> lock
    // critical section
}    // dtor -> unlock (ավտոմատ, նույնիսկ exception-ի դեպքում)
```

`std::lock_guard` -ը RAII է, lock ctor-ում, unlock dtor-ում: Չկա «մոռացած unlock» -> չկա deadlock:

5. RAII և Rule of Zero

Լավագույն RAII-ն այն է, երբ ձեռքով ոչ մի բան չես գրում: STL տիպերը (vector, string, unique_ptr) իրենք RAII են:

```
class Widget {
    std::vector<int> data;    // RAII ինքնին
    std::string name;    // RAII ինքնին
    std::unique_ptr<int> ptr;    // RAII ինքնին
};    // ոչ մի destructor պետք չէ -- Rule of Zero
```

Rule of Zero — եթե օգտագործում ես RAII տիպեր, ոչ մի ձեռքով destructor/copy/move պետք չէ; compiler-ի default-ները ճիշտ են:

6. Ինչու RAII-ն ամենակարևորն է C++-ում

- Exception-safe (stack unwinding -> ավտոմատ cleanup)
- Չկա մոնոացում (deterministic destruction)
- Չկա leak (memory, file, lock, socket)
- Չկա manual cleanup ամեն exit path-ում
- Deterministic (GC-ից տարբեր՝ կոնկրետ պահին ազատում)

7. Հարցազրույցի հարցեր և պատասխաններ

RAII ինչ է:

Resource Acquisition Is Initialization. resource-ը կապվում է object-ի կյանքին. վերցվում ctor-ում, ազատվում dtor-ում; scope-ի վերջում ավտոմատ cleanup:

Ինչո՞ւ է պետք:

Exception safety + չկա leak/մոնոացում. Ամեն exit path-ում (return/throw) resource-ը ավտոմատ ազատվում:

Ինչպես է աշխատում exception-ի դեպքում:

Stack unwinding -> C++-ն ավտոմատ կանչում է local object-ների destructor-ները -> cleanup:

RAII օրինակներ:

unique_ptr/shared_ptr (memory), lock_guard (mutex), fstream (file), vector/string:

RAII vs manual cleanup:

Manual' ամեն path-ում ձեռքով, հեշտ մոռանալ, exception-ի դեպքում leak. RAII' ավտոմատ, exception-safe, deterministic:

Rule of Zero-ի հետ կապը:

RAII տիպեր օգտագործելիս ոչ մի ձեռքով dtor/copy/move պետք չէ -> Rule of Zero:

Ամփոփում (Cheat Sheet)

RAII = Resource Acquisition Is Initialization

resource վերցվում -> constructor | ազատվում -> destructor

scope-ի վերջ / exception -> ավտոմատ cleanup (stack unwinding)

ՕՐԻՆԱԿՆԵՐ:

unique_ptr/shared_ptr -> memory

lock_guard/unique_lock -> mutex

fstream -> file

vector/string -> memory

ԱՌՎԿԵԼՈՒԹՅՈՒՆ:

exception-safe | չկա leak | չկա մոռացում | deterministic

RULE OF ZERO: RAII տիպեր -> ոչ մի ձեռքով dtor/copy/move